Event Tracing for Windows (ETW) Security: Attack Techniques and Defense Strategies

Prepared by: Birkan KES
Date: 25/08/2025

İçindekiler

INTRODUCTION 2
Introduction to ETW 3
ETW Architecture 4
Technical Details of the ETW Infrastructure 5
IBM's ETW Threat Model 6
ETW Attack Techniques 7
ETW Tampering 7
Environmental / Registry Attacks 9
Attacks Targeting User-Mode ETW Providers 10
Attacks Targeting Kernel-Mode ETW Providers 11
ETW Security Defense Strategies and Recommendations 12
Referanslar 15

INTRODUCTION

I started writing this article to explore an attack idea I had in mind. I wanted to share the information I learned and the attack techniques I encountered during this process.

In the world of cyber defense, visibility is the most important step for recording events correctly and detecting threats on time. This is where Windows Event Tracing for Windows (ETW) comes in. ETW is one of the most important telemetry mechanisms that records events from both user and kernel mode and provides data to EDR, SIEM, and antivirus solutions.

In the past, ETW manipulation was seen as a sophisticated method used only by APT groups. But this perception has changed. Today, methods like ETW API patching, syscall manipulation, kernel-based DKOM attacks, and direct deletion of log files are used not only in real attacks but also in test scenarios. Moreover, it is not only APT groups; ransomware gangs also target ETW now.

In this article, we will examine the structure of ETW, the bypass techniques used by attackers, and defense strategies. Based on what we learn, we will also try to develop our own recommendations and strategies.

Introduction to ETW

ETW (Event Tracing for Windows) is a built-in tracing system in the Windows operating system at the kernel level. It records events coming from both user-mode applications and kernel-mode drivers. Many EDR products and security tools collect telemetry through ETW, so attackers try to bypass or manipulate ETW to avoid detection.

The ETW architecture has four main components: Provider, Session, Controller, and Consumer.

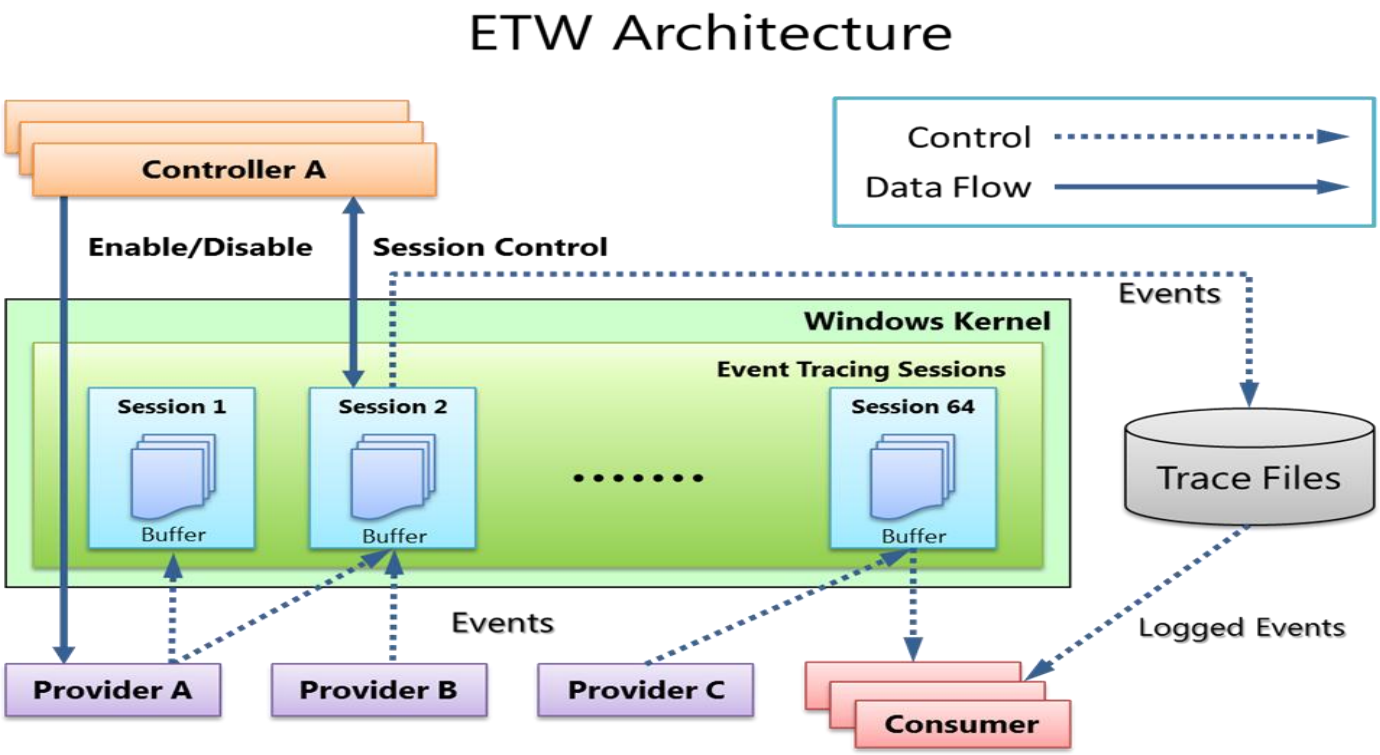
- **Provider:** These are components configured to generate events for tracing. A provider can be a user-mode or kernel-mode application, or a kernel driver itself. Providers register with ETW and mark specific points in the code to send event data. Each provider is identified by a GUID in an XML manifest file. The manifest also contains event templates, tasks, and defined data.

- **Session:** A session works as a kernel object. It collects events from one or more providers in kernel memory and writes them to disk as .etl files, or sends them to consumers in real time. A user can link multiple providers to a single session, so events from different sources are collected together. The Windows kernel supports up to 64 concurrent tracing sessions, with 8 system sessions (e.g., NT Kernel Logger, Global Logger) predefined. The NT Kernel Logger session is especially for recording kernel-level events (process/thread creation, DPC, interrupts, etc.) and only one instance can be active at a time

- **Controller:** The controller starts, stops, and configures ETW sessions. It decides which providers are enabled and sets session parameters. Tools like logman in the Windows command line or Windows Performance Recorder (WPR) act as controllers. When a controller runs (via EnableTrace or StartTrace functions), the session is created and can be stopped when needed. Windows also allows you to write your own controllers.

- **Consumer:** Consumers are applications that read and process collected events. A consumer can read a created .etl log file or connect in real time to an active tracing session. Examples include Event Viewer, Windows Performance Analyzer (WPA), or custom analysis tools. System components like Resource Monitor can also act as ETW consumers.

ETW Architecture



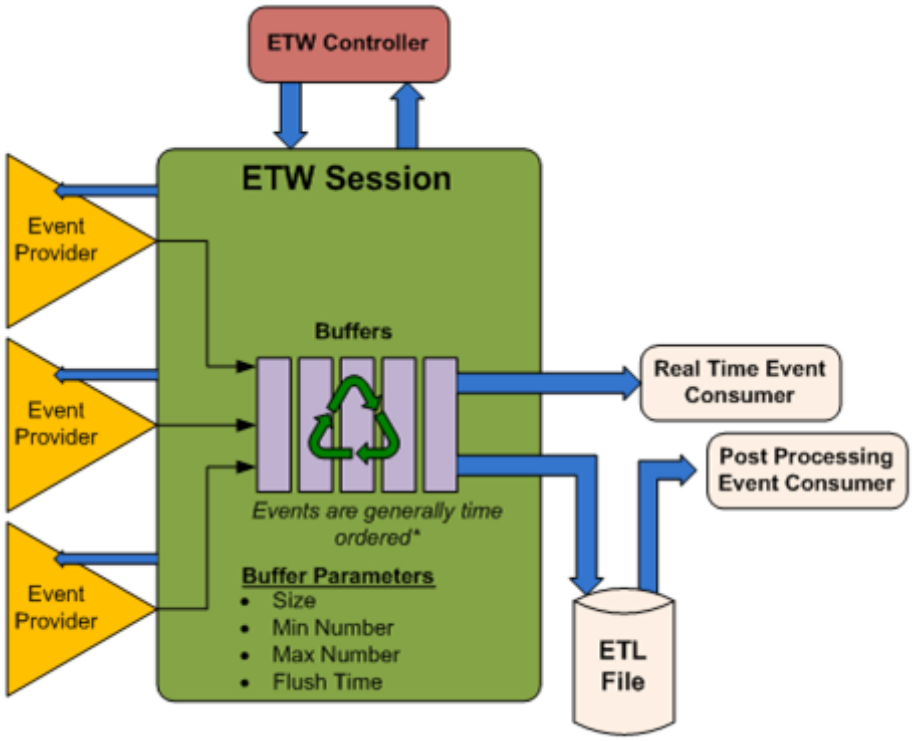
REF: https://learn.microsoft.com/en-us/windows-hardware/test/weg/images/weg-diagram-etw-architecture-four-components.png

Controllers enable providers, Event Tracing Sessions collect events in buffers, and then send them to files or consumers.

The event flow in ETW works as follows:

1. **Session Start:** The controller creates a new session and enables the relevant providers. At this stage, the session name, target .etl file, and the types of events to record are defined.
2. **Event Generation**: Enabled providers send events from their code using ETW API calls (like EventWrite or TraceLoggingWrite).
3. **Event Collection**: The kernel writes incoming events to memory areas in kernel mode. These are fixed-size areas reserved in advance. Data is stored in circular buffers or, when the buffer is full, it is written to disk.
4. **Data Transmission**: ollected event data can be used in two ways:
    - **Real-time transmission**: The kernel sends events directly to connected consumer processes (like WPA, Event Viewer).
    - **Log-based**: Events are first written to a .etl file on disk, which can be read later for analysis
5. **Usage and Analysis**: Consumer applications parse .etl files or process event streams in real time. For example, Event Viewer converts ETL to EVTX format, while Windows Performance Analyzer visualizes performance counters
6. **Real-time vs Log-based Recording:** Real-time mode is used for scenarios requiring low-latency monitoring (e.g., live performance tracking). Log-based recording keeps events permanently and allows offline analysis. When events accumulate in kernel buffers and reach a certain size, they are automatically written to .etl files.

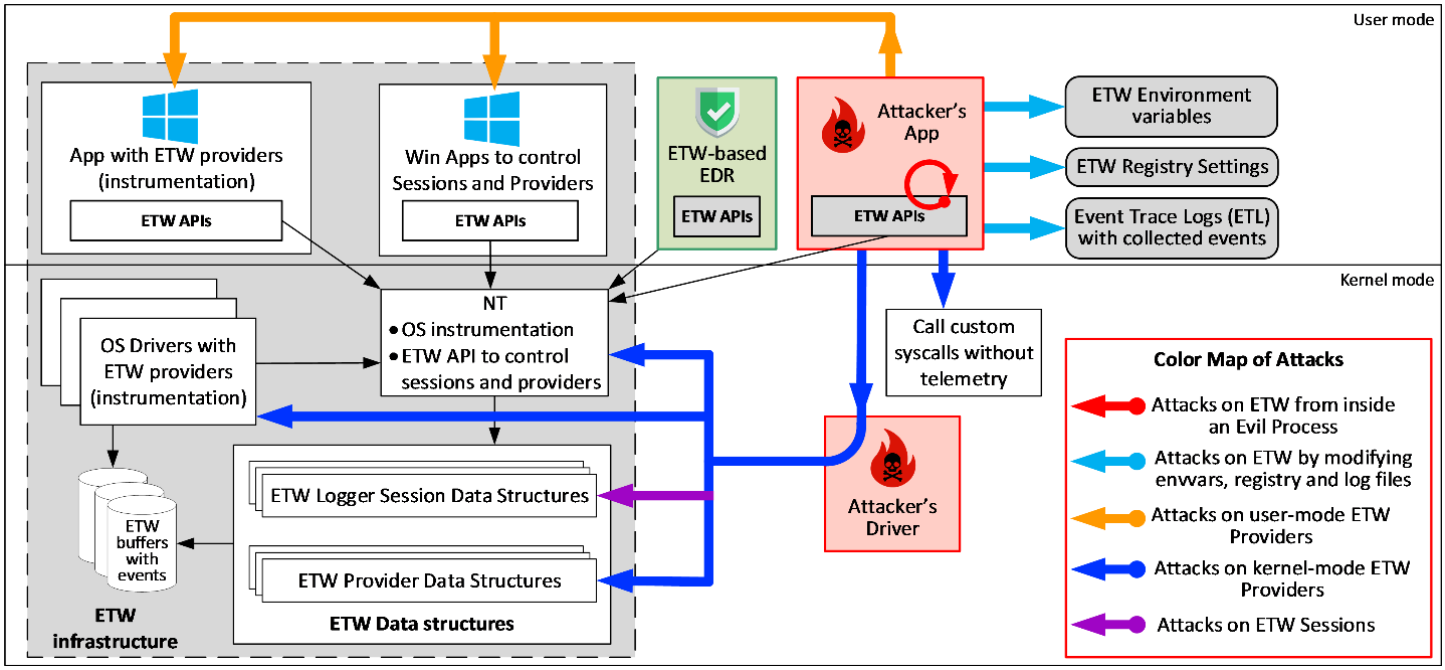
Technical Details of the ETW Infrastructure



REF:https://learn.microsoft.com/en-us/previous-versions/windows/desktop/xperf/images/image070.png

1. **Trace Session:** Each tracing session is identified by a name and managed with a 64-bit GUID. By default, Windows allows up to 64 concurrent sessions, with 8 system-level predefined sessions (e.g., NT Kernel Logger, Global Logger, Circular Kernel Logger). Users can also create their own sessions in user mode (Private Logger) or use customized sessions like AutoLogger for automatic system logging. Each session's security identifiers and configuration are stored in the registry.
2. **GUID and Manifests**: ETW providers are distinguished by GUIDs defined in manifest files. Similarly, each task, opcode, and channel has a unique ID. ETW manifests are XML-based and include the schema of event data, icons, and message templates. Using manual manifest compilation (mc.exe), *.bin and header files are generated, so applications can produce events statically or dynamically.
3. **ETL Files**: ETW data is usually written to binary log files with the .etl extension. These files contain raw events from kernel buffers and do not follow standard formats; they need tools like Windows Event Viewer, tracerpt.exe, or Windows Performance Analyzer to be analyzed. ETW events include rich metadata (timestamp, thread/process ID, CPU info, etc.) and are parsed after storage for analysis.
4. **Custom Sessions and Channels**: ETW also has the concept of channels; each provider can be linked to one or more channels (e.g., Admin, Operational, Analytic). System components like PowerShell and AMSI have their own Operational channels. Some channels are not included in default tracing sessions, so custom tracing sessions may be needed. Settings for these structures are stored in the registry under HKLM\SYSTEM\CurrentControlSet\Control\WMI\Autologger.

IBM's ETW Threat Model



REF: https://www.ibm.com/think/x-force/direct-kernel-object-manipulation-attacks-etw-providers

The threat model above shows the components that attackers can manipulate in **user mode** and **kernel mode**.

ETW Sağlayıcıları (Providers): Sistem ve uygulama olaylarını kaydeden bileşenler. Örn PowerShell, AMSI, Windows Kernel bileşenleri.

Oturumlar (Sessions): ETW sağlayıcılarının gönderdiği olayların toplandığı ve saklandığı yapılar. Örn: Default, Kernel-Logger vb.

ETW Kanalları (Channels): Olayların türüne göre ayrıldığı yollar.

ETW Attack Techniques

ETW Tampering

1) **Patching Application-Level ETW APIs:**

User-mode applications or system components call specific APIs to write events to ETW providers. Some of the most critical functions are in **ntdll.dll**, such as "**EtwEventWrite**, **EtwEventWriteFull**, and **EtwEventWriteTransfer"**. Attackers can patch these functions in memory (using hooking or inline patching) to interfere with ETW's event delivery.

One common method is to add small but effective instructions at the beginning of these APIs. For example, adding a "**RET"** or "**XOR EAX, EAX; RET"** instruction can make the call return immediately or indicate a failure, preventing the event from being recorded. This technique ensures that activities performed by the application or malware never reach Windows logging mechanisms, making them invisible to security products like EDR, SIEM, or antivirus.

ÖRNEK POC: Unhooking Event Tracing for Windows < BorderGate

2) **Disabling the Syscall Call:**

In Windows, ETW events are not delivered only through high-level APIs. These events eventually reach the kernel by triggering "**syscall"** functions (e.g., **NtTraceEvent**, **ZwTraceEvent**). These functions are located in **ntdll.dll** and allow user-mode applications to make a system call to record events in the ETW infrastructure.

An attacker can manipulate these syscalls to prevent the logs from reaching the kernel. This way, the relevant events never reach the ETW system. The transition can be blocked by replacing the first instruction of the embedded call stubs in the system with a "**RET"** instruction.

Important point:
Most EDR products monitor APIs like **EtwEventWrite**. However, if an attacker patches **NtTraceEvent** directly, these checks are bypassed, because most security products do not continuously monitor syscalls.

PoC and Ref: Bypassing ETW For Fun and Profit | White Knight Labs

3) **<u>Obfuscation, Encrypted, or Stealth Syscall</u>:**

Normally, when a process calls an API, the **call stack** can be tracked to see which functions were executed. However, APT groups use techniques to avoid security products by creating fake layers on the call stack or hiding the original call flow.

One important method in this context is the "**stealth syscall**" technique. Here, kernel calls (syscalls) are executed directly, encrypted, or obfuscated. The goal is to prevent security tools from detecting or tracking the raw syscall calls.

According to recent reports, these techniques usually combine three main methods:

- Call stack forgery
- ETW API hook bypass
- Encrypted or hidden syscall execution

This combination defeats traditional detection mechanisms and particularly renders user-mode ETW patch detection ineffective.

Environmental / Registry Attacks

1) **Manipulating or Deleting ETW Log Files:**

Attackers can target the files and folders where ETW tracing logs are stored. For example, in **Slingshot** APT attacks, ETW file extensions were changed to ".**tmp**" to avoid leaving traces on the system. Similarly, ETW logs can be hidden by moving them from their default locations or deleting them..

2) **Tampering with WMI and Logger Settings:**

ETW monitoring can be stopped by modifying environmental settings and services that control ETW sessions.

For example, **LockerGoga** used "**wevtutil"** to disable the **Microsoft-Windows-WMI-Activity Provider**, so WMI-based events like process creation were not logged.

Additionally, a malicious driver can modify ETW session configurations, such as the maximum number of providers, by changing **WMI_LOGGER_CONTEXT** structures. By altering these values, attackers can prevent new activities from being logged.

3) **Disabling Protections on System Drivers:**

Some important ETW providers run at **Protected Process Light** (PPL) level. Attackers can temporarily remove PPL protection to disable critical providers.

For example, sending signals to a PPL-protected process or using an unsigned driver to deactivate PPL can stop logging for protected ETW providers. This can prevent certain events generated by system services from being recorded.

Attacks Targeting User-Mode ETW Providers

1) **Provider-Based Hooking:**

An attacker can access the memory of a target application (using DLL injection or code injection) to interfere with the provider's functions (e.g., **EventWrite**, **TraceLoggingWrite**).

For example, an attacker may hook or patch functions like **EventWrite** or **TraceLoggingWrite** in a .NET application. As a result, even if these functions are called, the events either do not reach ETW or are redirected to an empty function.

2) **Removing ETW Provider Triggers:**

To disable a provider running in user mode, the attacker can inject into the process and deactivate the event corresponding to the provider's GUID.

For example, it is possible to disable a specific provider using **EnableTraceEx**/**EnableTraceEx2** calls. During these operations, EDR solutions usually scan process memory by default and may report anomalies.

Attacks Targeting Kernel-Mode ETW Providers

1) **WPP / Kernel Trace Hooks:**

Windows Performance Tracing (WPP) is a tracing mechanism used for Windows drivers and kernel components. Kernel ETW providers register their events to ETW through "**TraceLoggingRegister"** or "**EtwRegister"**. Normally, user-mode ETW providers send events using "**EventWrite"**, but in drivers, these functions are handled by the kernel

Attackers can load their own drivers to modify these providers. For example, they can interfere with kernel events registered via "**TraceLoggingRegister"** or patch "**EtwRegister"** objects. Even if the providers and sessions appear open, process or driver events may not be reported to ETW.

2) **Disabling Protected (Manifest-Based) Kernel Providers:**

The Windows kernel monitors some critical events (e.g., authentication, token operations, thread/process activities) using protected kernel ETW providers. These providers are "**manifest-based"** and hidden because only processes with special privileges can use them..

Access is usually limited to processes with **Protected Process Light** (PPL) status. An attacker can remove PPL using a custom driver and block kernel tracing providers, preventing the collection of "**Protected**" kernel events.

3) **Modifying Kernel ETW Tables via DKOM:**

ETW in the kernel is managed with structured tables:

- **WMI_LOGGER_CONTEXT** → controls active sessions
- **ETW_GUID_ENTRY** → records provider registrations
- **TRACE_ENABLE_INFO** → shows which events are active/inactive
- **TraceInformation table** → shows the state of kernel ETW operations in Windows 10/11.

These tables are stored in kernel memory and are normally inaccessible from user mode. If an attacker has kernel-level code execution, they can modify "**WMI_LOGGER_CONTEXT"** or other ETW structures using DKOM (**Direct Kernel Object Manipulation**).

For example, overwriting the **TraceInformation table** or setting it to "**NULL"** can interfere with all kernel ETW operations.

ETW Security Defense Strategies and Recommendations

1) Defense Against File and Log Manipulation

- **File Integrity Monitoring (FIM)** → Monitor the integrity of critical ETW files such as .etl, .tmp, %SystemRoot%\System32\winevt\Logs.
- **Sysmon + EDR Rules** → Track abnormal operations like renaming, deleting, or moving ETW files.
- **Disk Monitoring** → For attacks like Slingshot-style renaming, monitor for unexpected abnormal file types (e.g., .tmp) outside of ETW files.

2) Defense Against WMI and Logger Context Manipulation

- **WMI Event Subscription Monitoring** → Continuously monitor for disabled WMI providers or configuration changes
- **ETW Internal Checks** → Regularly validate WMI_LOGGER_CONTEXT memory and check session integrity.
- **Event ID 23-24** → Use event logs or Sysmon to track WMI activity; suspicious stop/start operations can act as triggers

3) Defense Against Protected Process Light (PPL) Bypass

- **Driver Signing Enforcement** → Do not allow unsigned drivers (use HVCI + Device Guard).
- **Kernel Debug Mode Off** → When kernel debug is enabled, attackers can bypass PPL more easily.
- **ETW Heartbeat** → Perform periodic checks to verify that critical ETW providers are running. For example, if expected events from critical protected ETW providers (like LSASS) do not appear, trigger an alert.

4) Defense Against User-Mode Hooking and GUID Manipulation

- **EDR Process Memory Scans** → Check if APIs like EventWrite or TraceLoggingWrite have been patched.
- **ETW API Call Heuristics** → Calls like EnableTraceEx2 are normally rare outside system processes. Detect suspicious activity
- **.NET / CLR Monitoring** → Monitor managed applications for anomalies in the AppDomain that may indicate ETW hook attempts.

5) Defense Against Kernel Trace Hook and DKOM Attacks

- **Kernel ETW Watchdogs** → Monitor runtime integrity checks of ETW tables (e.g., TraceInformation, EtwGuidEntry).
- **Hypervisor-Based Protection (HVCI, VBS)** → Prevent DKOM attacks using memory guards.
- **ETW Cross-Validation** → Compare kernel events with user-mode ETW logs to check consistency.

6) General Strategy

- **Redundancy Logging** → ETW should not rely on a single source. Correlate with Sysmon, EDR driver telemetry, network telemetry, etc.
- **Decoy Providers** → Add fake ETW providers like a honeypot to catch manipulation attempts.
- **Anomaly Detection (ML-based)** → Monitor for drops in normal ETW event rates.
- **Protect Critical Processes** → Use PPL and WDAC to enhance the security of critical providers.
- **Apply Multi-Layered Defense**: Combine kernel + API integrity + file/log monitoring + telemetry correlation

Recommended Unified Strategy

1. **Kernel ETW + PPL / VBS / HVCI** → Core defense, anti-tamper.
2. **ETW API & Session Integrity** → etect manipulation in user and kernel mode.
3. **File / Log Monitoring** → Catch abnormal file events using FIM + Sysmon.
4. **Driver Security** → Ensure signed drivers, Secure Boot, and patch management.
5. **Anomaly and Redundancy** → ML tabanlı event monitoring, cross-log validation, decoy providers
6. **Multi-Layered Defense** → Detect and prevent attacks using a combination of measures, not a single solution.

Attack Type	Technical Description	Prerequisites / Privileges	Impact Level	Defense / Recommendations
User-Mode ETW API Patch	Patching functions like EtwEventWrite, EventWriteEx, etc.	User-mode, usually no admin rights required	Only suppresses logs from the target process	API integrity checks, EDR hook verification, provider heartbeat, memory scans, telemetry health checks, cross-log correlation
Disable Syscall Body	Emptying syscall functions like NtTraceEvent, stealth syscall or stack trace hiding.	Kernel-mode access required	Disables logging of the relevant syscall across the system.	HVCI, PatchGuard, kernel memory integrity monitoring, stack trace analysis, behavior profiling
Obfuscation / Stealth Syscall	Hidden or encrypted access to syscall table; hides call stack.	User-mode but requires advanced exploit / EDR bypass	Process-level invisibility; events do not reach ETW logs.	Stack trace analysis, behavior profiling
Delete / Manipulate ETW Log Files	Delete, rename, or move .etl / .evtx log files.	Admin privileges required	Existing logs are deleted, moved, or hidden.	File integrity monitoring, Sysmon file deletion tracking, anomaly detection
WMI / Logger Configuration Tampering	Disable Autologger or provider configs in the registry.	Admin / SYSTEM privileges required	Certain channels stop producing logs system-wide.	WMI session integrity monitoring, registry/file monitoring, telemetry health check
Bypass System Driver Protections (BYOVD)	Load vulnerable drivers to remove ETW/EDR callbacks (PPL bypass).	Admin rights + driver loading (Ring-0)	Can completely silence EDR/ETW system-wide.	Driver blocklist, Secure Boot, HVCI, PatchGuard, WDAC, VBS, callback monitoring (PsSetLoadImageNotifyRoutine)
Provider-Based Hooking	Selectively disable certain ETW GUID providers; hook EventWrite or TraceLoggingWrite.	User-mode or kernel access depending on target	Only specific critical providers are suppressed (e.g., Defender, AMSI).	Provider integrity tests, API integrity checks, EDR memory monitoring
Remove ETW Provider Triggers	Nullifying EtwpActiveSystemLoggers or context pointers.	Kernel access (DKOM)	All active sessions silently disabled.	Memory integrity monitoring, kernel anti-tamper, DKOM detection
WPP / Kernel Trace Hooks	Hook kernel trace macros to manipulate output; patch TraceLoggingRegister / EtwRegister.	Kernel access required	Kernel debug/trace infrastructure is manipulated.	Kernel memory diffing, trace provider health checks, driver signing enforcement, HVCI/VBS, callback monitoring
Disable Protected Kernel Providers	Prevent logging of critical manifest-based providers; hidden ETW providers stopped via PPL bypass.	Kernel access required	Critical system logging (e.g., process creation) disabled.	Provider health checks, ETW session integrity, PatchGuard, PPL state monitoring, VBS, WDAC, Secure Boot
Modify Kernel ETW Tables via DKOM	Modify structures like _ETW_REG_ENTRY, _ETW_GUID_ENTRY in kernel memory; write to TraceInformation table.	Kernel access (Ring-0)	Entire ETW infrastructure disrupted; system becomes globally blind.	Kernel memory forensics, Secure Kernel (VBS / Isolated User Mode), anti-DKOM tools, decoy providers, cross-logging

Referanslar

From Cobalt Strike to Mimikatz: A Deep Dive into the SLOW#TEMPEST Campaign Targeting Chinese Users - Securonix

Tampering with Windows Event Tracing: Background, Offense, and Defense | by Palantir | Palantir Blog

New ETW Attacks Can Allow Hackers to 'Blind' Security Products - SecurityWeek

Remcos RAT Targets Europe: New AMSI and ETW Evasion Tactics Uncovered

Design issues of modern EDRs: bypassing ETW-based solutions

ETW Forensics - Why use Event Tracing for Windows over EventLog? - - JPCERT/CC Eyes | JPCERT Coordination Center official Blog

Instrumenting Your Code with ETW | Microsoft Learn

LockBit Ransomware Side-loads Cobalt Strike Beacon with Legitimate VMware Utility | SentinelOne

Unhooking Event Tracing for Windows < BorderGate

Anti-Tampering and Detection Evasion – Bitdefender TechZone

Bypassing ETW For Fun and Profit | White Knight Labs

Kernel ETW is the best ETW — Elastic Security Labs